CLoudflare Proj

Nice — since `wrangler dev` is working and you have the scaffold (`src/`, `wrangler.jsonc`, `tsconfig.json`, etc.), here's an updated, **practical 5-day plan** that plugs directly into your current project. Each day has concrete files to edit/create, commands to run, and exact checkpoint commits so you can show progress.

Day-by-day (5 day) starter plan — tailored to your `agent-worker` repo

Day 1 — End-to-end skeleton (mock LLM + simple UI)

Goal: Pages UI → Worker endpoint → mock planner response. End-to-end wire with `wrangler dev`.

Files to create / edit

- `src/index.ts` — Worker entry (route `/api/monitor` and `/api/chat`)
- `src/agent/planner.ts` — mock LLM wrapper (deterministic JSON)
- `public/index.html` — minimal chat/monitor UI (form + fetch)
- ensure `wrangler.jsonc` has routes (dev will handle)

Key code sketches

- `planner.ts` returns:

```
exportasyncfunctionplanFromChecks(checks,history) {
return { severity:"low", decision:"noop", reason:"all health
y", actions: [] };
}
```

- `index.ts` POST `/api/monitor` accepts `{ userId, url, interval_seconds }` and returns `{"monitorId":"...","status":"created"}`.

Commands

```
npm install# if not yet
wrangler dev
# open http://localhost:8787 (or pages URL if using Pages)
```

Checkpoint

```
git add . &&git commit-m"day1: E2E skeleton - worker endpoint
s + mock planner + simple UI"
```

Day 2 — Health checker + workflow design (local emulation)

Goal: Implement the actual health check logic and design the Workflow steps (you can emulate Workflows locally—create a worker-based orchestrator initially).

Files to create / edit

- `src/health/checker.ts` — `checkEndpoint(url)` returns `{ status, latency_ms, timestamp, region? }`
- `src/controller/orchestrator.ts` — small orchestrator that runs the agent loop (health_check → decide → act) for now (this emulates what Workflows will do)
- Update `src/index.ts` so `/api/monitor` starts orchestrator runs (store runs in-memory for now)

Checklist / behavior

- `checkEndpoint` uses `fetch(url, { cf: { scrapeShield: false }})` and times response
- The orchestrator performs X checks (configurable), calls `planner` with aggregated checks, and returns planner decision

Test scenarios

- Create local test endpoints (run a tiny Express server or use `httpbin.org/status/500`) and point the monitor to them
- Trigger: POST `/api/monitor` → see logs for checks and decision

Checkpoint

```
git commit-m"day2: health checker + local orchestrator (workf
low emulation)"
```

Day 3 — Durable Objects (state & incident model)

Goal: Replace in-memory state with Durable Objects for monitors, incidents and per-user memory.

Files to create / edit

- `src/do/AgentDO.ts` — Durable Object class handling monitors and incidents
- `wrangler.jsonc` — add DO binding (`"durable_objects": { "bindings":[{ "name":"AGENT_DO", "class_name":"AgentDO" }] }`)
- Update `src/index.ts` to `env.AGENT_DO.idFromName(userId)` and forward control to DO

DO responsibilities

- store `monitor` configs `{ id, url, interval, threshold }`
- store `incident` state `{ id, status: NEW|INVESTIGATING|ESCALATED|RESOLVED, history:[] }`
- expose debug read endpoints in dev (only)

Testing

- `wrangler dev` supports DOs via Miniflare — exercise create monitor, check DO state via debug route

Checkpoint

```
git commit-m"day3: durable objects for monitor & incident sta
te"
```

Day 4 — Integrate Workers AI (Llama 3.3 stub → real) + safe action

Goal: Wire the planner to Workers AI (or a clear, swap-in wrapper) and implement one safe action (dry-run notify).

Files to create / edit

- `src/agent/ai.ts` — wrapper `runPlanner(checks, history)` that calls `env.AI.run(...)` (make it fallback to mock when `env.AI` is not configured)
- Update `src/agent/planner.ts` to validate/parse JSON responses into typed `IncidentDecision`
- `src/actions/runner.ts` — implement `executeAction(action, dryRun=true)`; for now support `notify` (POST to mock Slack endpoint or write to DO audit log)

Prompt guidance (put in code / README)

```
You are an incident response planner. Input: recentChecks, hi
story, failure_threshold...
Respond ONLY with JSON:
{ "severity":"low|medium|high", "decision":"noop|retry|escala
te", "reason": "...", "actions":[{ "type":"notify", "args":
{...}}] }
```

Test

- Point monitor to a failing endpoint, run orchestrator:
  - planner returns `escalate` → `executeAction` writes to audit log / mock notify
- Validate DO incident state updated & audit entry present

Checkpoint

```
git commit-m"day4: integrate Workers AI wrapper + dry-run not
ify action"
```

Day 5 — Polish, tests, README & demo script

Goal: Make it presentable: unit tests, README (problem + architecture), short demo script and screenshots/gif.

Files to create / edit

- `README.md` — problem statement, architecture diagram (ASCII or embed), how to run locally, how to plug Workers AI credentials
- `src/tests/` — Vitest unit tests:
  - prompt-builder tests
  - planner JSON parser validation tests
  - `checkEndpoint` parsing (mock fetch)
- `demo/` — `demo.sh` script to run local test server toggling 200/500, and `curl` commands to start monitor
- Add a `/metrics` debug endpoint in `src/index.ts` showing counts: checks run, LLM calls, incidents created

Polish

- Short `architecture.png` (optional) or ASCII diagram in README
- Add explanation: why DOs vs KV; why Workflows; failure modes; cost estimate (LLM calls/incidents)
- Record a 30–60s screen capture or create screenshots for your README

Final commit

```
git tag v0.1
git commit-m"day5: polish, tests, README, demo script"
git push--tags
```

Extra practical tips & commands (right now)

- Run dev: `wrangler dev`
- To test DO behavior locally with Miniflare and faster cycles, you can add `"dev": { "local": true }` in config or run `npx miniflare --wrangler-config wrangler.jsonc` if needed.
- Fast test command for a monitor (example):

```
curl-X POST http://localhost:8787/api/monitor \
-H"Content-Type: application/json" \
-d'{"userId":"alice","url":"http://localhost:3000/down","inte
rval_seconds":30,"failure_threshold":2}'
```

- Use `TEST_MODE` env var to shorten sleep/delay times in orchestrator for local testing:

```
constDELAY=process.env.TEST_MODE?1000:120000;// 1s vs 2min
```

Minimal JSON shapes to lock in now (copy these into your code)

Monitor Config

```
typeMonitorConfig= {
  monitorId:string;
  userId:string;
  url:string;
  interval_seconds:number;
  failure_threshold:number;
};
```

Check Result

```
typeCheckResult= {
  timestamp:string;
  region?:string;
  status:number;
  latency_ms?:number;
  error?:string;
};
```

Planner Output

```
typeIncidentDecision= {
  severity:"low"|"medium"|"high";
  decision:"noop"|"retry"|"escalate";
  reason:string;
  actions:Array<{ type:string; args?:Record<string,any> }>;
}
```